

Design the NANO-8: An 8-Bit Instruction Set Architecture

You are the lead ISA architect at MicroNano Corp. The silicon is already committed. Your job is to decide what the processor can do, and then prove your design works.

The Scenario

MicroNano Corp is producing the **NANO-8**, an ultra-low-power 8-bit microcontroller for embedded sensing applications — medical patches, environmental sensors, tiny IoT nodes. The chip’s silicon layout is frozen. What you *can* define is the Instruction Set Architecture: the operations the processor will support, how they are encoded, and what constraints programmers must work within.

Hard Constraints

Parameter	Value
Instruction word width	Exactly 8 bits
General-purpose registers	4 (R0, R1, R2, R3)
Data memory	16 bytes (addresses 0x0–0xF)
Program memory	64 instructions (6-bit PC)
Data width	8-bit unsigned integers (0–255)
Flags register	Z (zero) and C (carry)

The central design tension

With only 8 bits per instruction and 2 bits per register field, your "bit budget" is tight. An instruction that specifies two registers already uses 4 bits — leaving only 4 bits for everything else (opcode, immediate, address). A 4-bit opcode allows only 16 distinct instructions. An immediate value field of only 2 bits means your constants are limited to 0, 1, 2, or 3. Every decision you make involves tradeoffs. Welcome to ISA design.

Mandatory Capability Requirements

As long as you are able to write assembly code for all the given pseudo codes in this assignment your ISA is fine.

Task Assembly Programming in Your ISA 30 points

Write assembly programs in **your own NANO-8 ISA** for the following three tasks. For each, annotate every instruction with a comment. If your ISA cannot natively support an operation, describe the issue, propose a fix, and update accordingly.

Program 1 — "The Warm-Up" (8 points)

$R1 = 3$, $R2 = 5$, $R3 = R1 + R2$, $MEM[3] = R3$

Assume the constants 0, 3 and 5 are available at data memory locations 0, 1 and 2 respectively.

Write the shortest correct assembly sequence for this in your ISA.

Program 2 — "The Accumulator" (12 points)

```
sum = 0 // initialize accumulator
i = 1 // loop counter starts at 1
while i <= 8: sum = sum + i // accumulate
i = i + 1 // increment counter
MEM[1] = sum // store result (should be 36)
```

At the end, calculate the **dynamic instruction count** (total instructions executed across all 8 loop iterations plus setup and end).

Program 3 — "The Sentinel" (10 points)

```
sensor_val = MEM[2] // read sensor
threshold = 7
if sensor_val > threshold: MEM[3] = 1 // alert!
else: MEM[3] = 0 // normal
```

Design Challenge — the comparison problem

Your ISA has a Z flag (set when result = 0) and a C flag (set on carry/borrow). How do you implement "if A > B" with only these?

Hint: compute A - B. If A > B, the subtraction produces a positive result (no borrow, C = 0 in subtraction-with-borrow convention). If A ≤ B, there's a borrow (C = 1). This means you might not need a dedicated CMP instruction — a SUB that sets flags could be enough, if your ISA defines flag behavior correctly. Does yours? If not, you may need to add a comparison instruction.

Write the complete assembly program with annotations.

Task Performance & Design Analysis 20 points

4A. Code Density (10 points)

Write down the static instruction count and dynamic instruction count for all the three programs for your ISA.

4B. The Immediate Ceiling (5 points)

What is the largest constant your ISA can load into a register using a *single* instruction? (You can not use data memory for loading constants, there must be a provision to load arbitrary

constants in your ISA)

Now: a temperature sensor returns values in the range 0–200. Your program needs to load the threshold value 150 into a register. How many NANO-8 instructions does this require? Describe the technique (hint: if you can load small constants, can you shift and add to build larger ones?).

4C. Opcode Utilization (5 points)

Looking at all three programs combined, list every instruction in your ISA and mark whether it was used. What percentage of your defined instructions were actually needed to complete all three programs? If some instructions were never used, could you have designed a simpler ISA — with fewer opcodes — that still completed all three tasks? Would removing those instructions free up opcode space for something more useful?

Bonus NANO-8 Version 2.0 — One High-Impact Extension +10 bonus points

Based on the pain points you discovered, could you extend your ISA to the maximum capacity so that it supports as much features as it can, given the constraints? (There are no correct answers to this?)

What You're Being Graded On

Criterion	What we look for
Format design	Formats are unambiguous, documented clearly
Instruction set	Every instruction's 8-bit encoding is shown explicitly.
Assembly programs	Programs correctly implement the pseudo-code. Instructions are annotated.
Performance analysis	Metrics are calculated correctly
Bonus	Proposed extensions demonstrably improves any specific program.

Grading philosophy

There is **no single correct ISA** for the NANO-8. An ISA with 8 very general instructions can complete all three programs just as correctly as one with 15 specialized instructions. You are graded on the quality of your reasoning, the internal consistency of your design, and your ability to identify and explicitly address the tradeoffs you made. A student who designs a clearly limited ISA but correctly analyzes its limitations will score higher than one who presents a seemingly complete ISA without understanding why it works.